CS1200: Intro. to Algorithms and their Limitations Profs. Anshu and Nesson

Lecture 2: Computational Problems and their Complexity

Harvard SEAS - Fall 2026 *2026-09-09*

1 Announcements

- Anurag's upcoming OH: today 2:15-3:15 SEC 3.323.
- Pset 1 released.

2 Recommended Reading

- Hesterberg–Vadhan, Sections 2.5, 3.1-3.2, 3.6, Appendix A.3
- CLRS 3e Ch. 2

3 Computational Problems

In the theory of algorithms, we want to not only study and compare a variety of different algorithms for a single computational problem like SORTING, but also study and compare a variety of different computational problems. We want to classify problems according to which ones have efficient algorithms, which ones only have inefficient algorithms, and which ones have no algorithms at all. We also want to be able to relate different computational problems to each other, via the concept of *reductions*. All of this requires having an abstract definition of what a computational problem is, and what it means for an algorithm to solve a computational problem.

Definition 3.1. A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example 3.2. Let's put the SORTING problem into the formalism of Definition 3.1.

- $\mathcal{I} = \mathcal{O} =$
- $f(x) =$

Note that there can be multiple valid outputs, which is why $f(x)$ is a set. For instance, in the example of SORTING above, if the input array x has pairs (K_i, V_i) and (K_j, V_j) with $K_i = K_j$ but $V_i \neq V_j$, then there are multiple valid answers.

The following definition is an informal way to describe an algorithm.

Informal Definition 3.3. An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.

We will be more precise about this definition in a few weeks, but for now you can think of a "procedure" as something that you can write as a computer program or in pseudocode like we have seen for sorting algorithms.

The next definition tells us what it means to "solve" a computational problem.

Definition 3.4. Algorithm A *solves* computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if the following holds:

Remarks.

- An algorithm A is supposed to have a fixed, finite description; and it should correctly solve the problem Π for *all* of the (infinitely many) inputs in the set $\mathcal{I}$. For instance, all the sorting algorithms discussed in Lecture 1 were described fairly concisely.

  In contrast, it would not qualify as an "algorithm" to list all (infinitely many) arrays of pairs and specify, for each of them, a sorting of it.

- Our proofs of correctness of sorting algorithms are exactly proofs that the algorithms satisfy Definition 3.4. This holds generally and we will frequently return to this definition throughout the course.

A fundamental point in the theory of algorithms is that we distinguish between computational problems and algorithms that solve them. A single computational problem may have many different algorithms that each solve it (or even no algorithm that solves it!), and our focus will be on trying to identify the most efficient among these.

4 Measuring Efficiency

To measure the efficiency of an algorithm, we consider how its computation time *scales* with the size of its input. To do so, we first define a size parameter, a function $\mathsf{size} : \mathcal{I} \to \mathbb{R}^{\geq 0}$. For example, in sorting, we typically let $\mathsf{size}(x)$ be the length n of the array x of key-value pairs. Sometimes we define (and measure the efficiency of algorithms in terms of) multiple size parameters: for instance, in the upcoming Sender-Receiver Exercise on `SingletonBucketSort`, we will measure the input size as a function of both the array length n and the size U of the universe of possible keys.

Informal Definition 4.1 (running time)**.** For an algorithm A, an input set $\mathcal{I}$, and input size function $\mathsf{size} : \mathcal{I} \to \mathbb{N}$, the *(worst-case) running time* of A is the function $T : \mathbb{R}^{\geq 0} \to \mathbb{N}$ given by:

$$T(n) =$$

This is referred to as *worst-case* running time because we take the *maximum* runtime over all inputs of size at most n. A couple of choices in the definition of $T(n)$ may seem unusual, but turn out to be technically convenient:

- We take the maximum over inputs of length *at most* n, not just equal to n.
- The domain of T is $\mathbb{R}^+$, not just $\mathbb{N}$.

For flexibility, we also introduce a variant definition that considers only inputs of size equal to n.

Informal Definition 4.2 (running time variant)**.** For an algorithm A, an input set $\mathcal{I}$, and input size function $\mathsf{size} : \mathcal{I} \to \mathbb{N}$, the *(worst-case) running time for fixed-size inputs* of A is the function $T^= : \mathbb{N} \to \mathbb{N}$ given by:

$$T^=(n) =$$

Note that for all $n \in \mathbb{N}$, $T^=(n) \leq T(n)$ and these are usually equal.

Remarks.

- *Basic operations:* Basic operations are arithmetic on individual numbers, manipulation of pointers, and writing/reading individual numbers to/from memory.

  Q: Should the Python function `A.sort()` count as a basic operation?

- *Worst-case runtime:*

- *Other notions of efficiency:*

To avoid having our evaluations of algorithms depend on minor differences in the choice of "basic operations" and instead reveal more fundamental differences between algorithms, we generally measure complexity with asymptotic growth rates, using big-O notation and variants.

5 Merge Sort

Finally, you may have already seen the following even more efficient sorting algorithm, `MergeSort`. The idea is to recursively sort each half of the array and then efficiently "merge" the two sorted halves into a single, sorted array:

We may not cover this in lecture (depending on time), since most of you have already seen it in CS 50. But you should review it from the textbook on your own!

```
   MergeSort(A):
   Input      : An array A = ((K_0, V_0), ..., (K_{n-1}, V_{n-1})), where each K_i ∈ ℝ
   Output     : A valid sorting of A
0  if n ≤ 1 then return A;
1  else
2  |  i = ⌈n/2⌉
3  |  B = MergeSort(((K_0, V_0), ..., (K_{i-1}, V_{i-1})))
4  |  B' = MergeSort(((K_i, V_i), ..., (K_{n-1}, V_{n-1})))
5  |  return Merge (B,B')
```

Algorithm 5.1: `MergeSort()`

We omit the implementation of `Merge`, which you can find in the textbook.

Theorem 5.1. `MergeSort` *correctly solves the* Sorting *problem. That is, for every input array* A, `MergeSort(`A`)` *returns a valid sorting of* A.

Naturally, the proof of this theorem will rely on the correctness of `Merge`, whose proof we leave as an exercise:

Lemma 5.2. *If* B *and* B' *are sorted arrays, then* `Merge`(B, B') *is a valid sorting of* $B \circ B'$.

Proof Sketch of Theorem 5.1. Like with `InsertionSort`, this is a proof by induction, but we use *strong* induction. (Why?)

Here, the statement we will prove by (strong) induction is simpler than for `InsertionSort`. It is simply

$$P(n) = \text{“}\texttt{MergeSort} \text{ correctly sorts arrays of size } n.\text{”}$$

□

6 Singleton Bucket Sort

Let's precisely define the computational problem of sorting on a finite universe.

Input: A universe size $U \in \mathbb{N}$ and an array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in [U]$

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a valid sorting of A.

Computational Problem Sorting On Finite Universe()

We will prove:

Theorem 6.1. *There is an algorithm for* Sorting On Finite Universe *on arrays of size* n *and key-universe size* U *with (worst-case) running time* $O(n + U)$. *One such algorithm is called* `SingletonBucketSort`.

The `SingletonBucketSort` algorithm is also known as "Pigenhole" sorting algorithm in some texts.

`SingletonBucketSort(`U`,`A`)`:

Input : A universe $U \in \mathbb{N}$ and an array $A = ((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each $K_i \in [U]$

Output : A valid sorting of A

0 Initialize an array C of length U, such that each entry of C is the start of an empty linked list.;

1 **foreach** $i = 0, \ldots, n-1$ **do**

2 | Insert (K_i, V_i) into the linked list $C[K_i]$ as the new head.

3 Form an array A' that contains the elements of $C[0]$, followed by the elements of $C[1]$, followed by the elements of $C[3]$, ...

4 **return** A'

Algorithm 6.1: `SingletonBucketSort()`

Example: $U = 5$ and $A = ((2, a), (3, b), (0, c), (4, d), (3, e), (3, f), (0, g))$.